Capacity, Not Associativity: A Reproduction Study of a Seamless Data-Cache Extension for Multi-Core RISC-V

Krishna Gorai
Indian Institute of Technology Mandi
Mandi, Himachal Pradesh, India

Bikram Paul
Indian Institute of Technology Mandi
Mandi, Himachal Pradesh, India

***Abstract*—We reproduce, from scratch and in open-source SystemVerilog, the snoop-coherent private L1 data-cache extension proposed by Kamaleldin *et al.* for FPGA-based multi-core RISC-V systems. The reproduction integrates an unmodified CV32E40P core, a write-through no-allocate data-cache unit, a snooping invalidation bus and an AXI4 crossbar into a four-processing-element system, and implements it on a Zynq UltraScale+ ZCU104. Reproducing the original's hit-rate figure required no change to the hardware but a specific change to how the benchmark kernels write their results, which we report as a reproducibility finding. Our principal result concerns the original's FFT curve: a controlled sweep that holds capacity fixed while doubling associativity, and then the reverse, shows the reported flat curve does not follow from the stated 2 KiB cache. Capacity recovers the curve completely while associativity recovers a sixth of it at $N{=}512$ and almost nothing at $N{=}1024$, a ratio of $6.6{:}1$ widening to $44{:}1$. We further show the extension's benefit is contingent on reuse and memory latency rather than intrinsic: on a streaming kernel at low memory latency the extension is slower than no cache at all, and the same kernel reaches $2.285\times$ when only the latency is raised. The complete design, data and regeneration scripts are public.**

***Index Terms*—RISC-V, cache coherence, snooping protocols, FPGA, reproducibility**

I. Introduction

Attaching a data cache to a soft RISC-V core without modifying the core is an attractive proposition for FPGA-based multi-core systems: the core remains a verified, unmodified IP block, and coherence is handled entirely in hardware. Kamaleldin *et al.* [1] propose exactly this, a private L1 data-cache sub-system attached to an in-order RISC-V core through its existing memory interface, with a snooping invalidation bus maintaining coherence and no software involvement.

We set out to reproduce that design in open-source RTL. This report presents the results obtained so far. Our contributions are:

- A from-scratch, open-source SystemVerilog implementation of the sub-system, integrated into a four-PE RISC-V system and implemented on a ZCU104, with the non-coherent baseline built from the same sources so the difference between them is attributable.
- A controlled measurement showing that the flat FFT hit-rate curve reported in the original does not follow from the cache configuration the original states, and that the effect is one of capacity rather than conflict (Section V-C).
- A measurement bounding the extension's benefit: it is contingent on reuse and memory latency, and is negative on a streaming kernel at low latency (Section V-D).
- A reproducibility finding: the original's execution-time figure cannot be reproduced with kernels that accumulate results in a register, for a reason specific to write-through no-allocate caches, and requires no hardware change to resolve (Section V-B).
- Six protocol and implementation hazards encountered in building the design that the original's description does not cover (Section VI).

II. The Design Under Reproduction

The original attaches a Data Cache Unit (DCU) between an in-order RISC-V core and the memory system. The DCU is a private, write-through, no-allocate L1 data cache. Coherence is maintained by a snooping invalidation bus: a write broadcasts an invalidation to the other DCUs, which drop any matching line. Because the policy is write-through, memory always holds the current value and no dirty-line writeback or ownership state is required. The core is unmodified and unaware of the cache, which is the origin of the "seamless" claim.

Our reproduction targets the stated configuration: a 2 KiB DCU, two-way set associative, 64 sets of 16-byte lines, one per processing element, with four PEs in the system.

III. Implementation

Each processing element comprises an unmodified CV32E40P core (pinned as a git submodule, with `COREV_PULP`, `COREV_CLUSTER` and the FPU disabled), a private instruction tightly-coupled memory, an Instruction-Bridge and a Data-Bridge. The core issues ordinary loads and stores; address decoding in the Data-Bridge routes coherent traffic to the DCU and everything else past it.

The four PEs meet the shared memories through a real AXI4 crossbar rather than a point-to-point arbiter, because the original lists the interconnect as a separate resource line and a simplified fabric would not be comparable. Each PE presents three masters onto three slaves. AXI4 rather than AXI4-Lite is deliberate: a line fill is one INCR burst of four beats, so a

miss pays one arbitration and one address phase rather than four.

A non-coherent baseline is built from the same sources with a single parameter changed, substituting a bypass shim for each DCU. Every comparison in this report is between two builds that differ in that one parameter and nothing else.

IV. Verification

The design is verified by self-checking testbenches at three levels: unit (the DCU and the AXI fabric in isolation), subsystem (four DCUs and the snooping bus against a version-monotonicity coherence checker), and system (the full four-PE design running bare-metal kernels). At the system level the coherent and non-coherent builds run identical binaries and are required to produce identical checksums, so any coherence failure appears as a value mismatch rather than as a performance difference.

V. Results

A. Functional reproduction

The four-PE system passes 273 self-checking assertions on a parallel bring-up kernel in both the coherent and non-coherent configurations, with identical results. Cache statistics are reported in the split the original's hit-rate figure uses: on this kernel the coherent build records a 85.35 % read hit rate and a 93.77 % write hit rate.

B. Reproducing the execution-time figure

The original reports execution-time reductions of approximately 40, 40 and 23 % for its three kernels. Our first attempts fell far short, and the cause was neither the cache nor the baseline, both of which were already correct.

Under a write-through no-allocate policy, a store to a line not currently resident neither allocates nor hits. A kernel that accumulates its result in a register and writes it once at the end therefore produces a write hit rate pinned near zero regardless of how well the cache is working, because the single write is always to a line the cache has no reason to hold. The kernels must accumulate *in memory*, and the matrix multiplication must use i-k-j ordering so that the accumulating row stays resident.

With that change and appropriately sized working sets, and *no change to the RTL*, we obtain reductions of 38.4, 48.0 and 18.5 % against the original's 40, 40 and 23 %. We report this because it is invisible from the original's description: the figure is reproducible, but only with a benchmark writing discipline the paper does not state.

C. Capacity versus associativity

The original's FFT curve is reported as flat across problem sizes at the stated 2 KiB DCU. We do not observe a flat curve.

The per-PE FFT working set is $8N$ bytes, so $N{=}512$ is the first size that exceeds a 2 KiB cache. It is also the first size at which the butterfly half-span reaches the set-index period, so the two operands of a butterfly alias onto one set. Capacity pressure and conflict aliasing arrive together and a single run cannot distinguish them. Four configurations can: holding capacity at 2 KiB while doubling associativity isolates conflict, and holding associativity at two ways while quadrupling capacity isolates capacity.

TABLE I
All sixteen capacity runs: FFT on the coherent build, four cache configurations at four problem sizes. Read hit rate above, execution cycles below. All four configurations are bit-identical at the two control sizes, so only the parameter under test moves. Regenerated by `scripts/sweep_capacity.sh`.

N	Working set	2-way 2 KiB	4-way 2 KiB	2-way 8 KiB	2-way 16 KiB
Read hit rate (%)					
128	1 KiB	96.8	96.8	96.8	96.8
256	2 KiB	97.2	97.2	97.2	97.2
512	4 KiB	**66.0**	70.8	**97.5**	97.5
1024	8 KiB	**57.8**	58.7	**97.7**	97.7
Execution cycles					
128	1 KiB	37 891	37 891	37 891	37 891
256	2 KiB	85 093	85 093	85 093	85 093
512	4 KiB	244 338	234 999	184 538	184 538
1024	8 KiB	574 366	569 565	415 782	415 782

Table I gives the result. At the two sizes whose working set fits, all four configurations are bit-identical, which establishes that nothing but the parameter under test is moving. At $N{=}512$ the stated configuration loses 31.2 points of read hit rate. Doubling associativity returns 4.8 of them; quadrupling capacity returns 31.5, restoring the pre-overflow rate. At $N{=}1024$ associativity returns 0.9 points and capacity returns 39.9.

Capacity therefore outweighs associativity by 6.6:1 at $N{=}512$ and 44:1 at $N{=}1024$. The ratio widens with the working set because additional ways cannot help once nothing fits. In execution time the stated configuration costs 24.5 % at $N{=}512$ and 27.6 % at $N{=}1024$ relative to a cache that holds the working set.

The observation that makes this a capacity argument rather than an arbitrary one is the final column: 16 KiB is bit-identical to 8 KiB at every size. Capacity helps until the working set fits and then stops entirely. A conflict-dominated effect would not saturate in that way.

We therefore report that the flat FFT curve does not follow from a 2 KiB DCU. A cache that holds the $8N$-byte per-PE working set produces it; the stated configuration produces a 31-point collapse at $N{=}512$ and a 39-point collapse at $N{=}1024$.

D. What the extension is actually worth

The reported benefit of a cache extension is not a property of the extension alone. The DCU introduces a second pipeline stage, so every access costs one cycle it did not cost before, plus a snoop arbitration; the non-coherent path pays neither. That overhead is fixed. The benefit, by contrast, has two terms: how often a line is touched again before eviction, and how much memory latency each of those hits avoids. Neither term alone predicts the outcome.

TABLE II
ALL TWELVE LATENCY RUNS, EACH A PAIR OF BUILDS EXECUTING THE SAME BINARY. CYCLE COUNTS ARE OF THE SLOWEST PROCESSING ELEMENT. SPEEDUP IS THE NON-COHERENT COUNT OVER THE COHERENT ONE, SO A VALUE BELOW ONE MEANS THE EXTENSION COSTS MORE THAN IT SAVES. REGENERATED BY SCRIPTS/SWEEP_MEMLAT.SH.

	MemLat 2			MemLat 8			MemLat 20		
Kernel	Coherent	Baseline	Speedup	Coherent	Baseline	Speedup	Coherent	Baseline	Speedup
memcpy (16 KiB)	21 010	20 493	**0.975**	27 151	40 975	1.509	39 439	90 129	2.285
FFT (N=256)	85 093	113 092	1.329	88 159	199 829	2.267	94 305	417 278	4.425
matmul (64^2)	1 337 850	2 191 981	1.638	1 724 046	5 288 075	3.067	2 544 265	11 627 128	4.570
conv2d (64^2)	168 177	311 454	1.852	177 847	735 126	4.133	202 825	1 610 668	**7.941**

Table II sweeps both builds over three memory latencies. Speedup is monotonic in latency for every kernel, and the ordering between kernels is preserved at all three latencies, which indicates the two terms act independently rather than interacting.

The first row is the important one. At a memory latency of two cycles the extension makes memcpy *slower*: 21 010 cycles against the baseline's 20 493, a speedup of $0.975\times$. The same kernel, unchanged, reaches $2.285\times$ at a latency of twenty. Reuse is constant across those three runs, so latency alone moves the result across the break-even line. A cross-kernel comparison cannot establish that, because reuse and latency vary together; a single kernel crossing over does.

The mechanism is visible in the cycle counts rather than the ratios. Raising latency from 2 to 20 costs the non-coherent conv2d build 417 % more cycles and the coherent build 21 %; for FFT the figures are 269 % and 11 %. The cached builds barely register the change. What the extension does is convert a latency-bound workload into a compute-bound one, and the speedup therefore measures chiefly how latency-bound the workload was to begin with. Where there is no reuse and no latency to hide, as in memcpy at MemLat 2, there is nothing to convert and the overhead is all that remains.

We note also that the growth is not unbounded. matmul rises $1.638 \rightarrow 3.067 \rightarrow 4.570$, decelerating as the cached build's residual misses come to dominate its own runtime, at which point further latency penalises both builds proportionally.

This bounds the original's headline result rather than contradicting it. The reductions the original reports are reproducible, but they are contingent on the memory latency of the platform, and a system with fast shared memory and a streaming access pattern can lose from the extension outright.

E. FPGA implementation

Both configurations were synthesised and implemented on a Zynq UltraScale+ XCZU7EV (ZCU104) in a non-project Vivado 2025.1 flow, and independently through a managed project, at a 75 MHz system clock derived from the 300 MHz board clock by an integer divider.

Table III gives the per-module breakdown. The cache extension costs 5 117 LUTs and 3 487 flip-flops over the non-coherent baseline (26 635 against 21 518 LUTs, 15 155 against 11 668 flip-flops), together with 20 additional block RAM tiles and no additional DSPs. Two observations are worth drawing

TABLE III
POST-IMPLEMENTATION RESOURCES, COHERENT BUILD, XCZU7EV AT 75 MHZ. REGENERATED BY SCRIPTS/TABLE1.PY.

Module	×	LUTs	FFs	BRAM
Processing element	4	17 837	9 948	32
RISC-V core	4	16 937	8 404	0
ITCM + I-Bridge	4	253	36	32
Data-Bridge	4	319	32	0
AXI4 masters	12	328	1 476	0
Cache extension	1	**5 322**	**4 118**	**20**
Data Cache Unit	4	5 019	4 000	20
controller	4	1 697	2 848	0
tag RAM	4	975	96	4
data RAM	4	2 364	1 056	16
Snoopy bus	1	303	118	0
AXI4 crossbar	1	2 127	279	0
Shared data memory	1	724	147	64
Shared instruction memory	1	506	86	8
Control region	1	114	259	0
Total system		26 635	15 155	124

TABLE IV
POST-IMPLEMENTATION TIMING, XCZU7EV, 75 MHZ TARGET.

	Coherent	Baseline
Worst negative slack (setup)	+0.880 ns	+0.929 ns
Worst hold slack	+0.003 ns	+0.010 ns
Failing endpoints	0	0
Timing constraints met	yes	yes

out. First, the extension is 20 % of system LUTs against the four cores' 64 %, so it is not the dominant area cost. Second, within the DCU the controller outweighs the storage, 1 697 LUTs of control against 975 of tag RAM, because at 2 KiB the arrays are nearly free and the coherence logic is not.

Timing is given in Table IV. Both configurations meet every constraint. Reaching that state required one change to the board wrapper: the single register driving every asynchronous reset in the system fans out to approximately fifteen thousand clear pins, and the removal check on reset release failed on routing distance alone, with 97 % of the worst path in wire. Constraining that register's fan-out so the tool replicates it removes the violation. We note that before this change the coherent build met timing by 12 ps in one placement and failed the same check in two others: a result that was a property of the placement rather than of the design.

F. Post-implementation verification

The implemented design was verified by simulating the placed-and-routed netlist. A board-level testbench drives only the differential board clock, the reset button and the run switch, and loads nothing: the program reaches the design as block RAM initialisation contents established at synthesis. All four processing elements complete and report success, and the netlist tracks the register-level simulation cycle for cycle, confirming that implementation introduced no functional divergence.

VI. Protocol and Implementation Hazards

Building the design surfaced hazards that its published description does not cover. We report them because they are the substance of making such a design correct.

1) **Shared array read port against a stalled stage.** The tag and data array read address is driven by the first pipeline stage; while the second stalled on a miss, the first stage's differing index displaced the data the second was still evaluating. Every hit decision taken after a stall used the wrong set.
2) **Coherence lock released before the write lands.** Releasing the single-writer lock when the invalidation is *granted* is insufficient. A remote read granted between that point and the write-through actually reaching memory fetches the stale value and caches it, and no further invalidation is ever sent. The lock must be held until the write lands.
3) **A failed store-conditional releasing another core's reservation.** With one link register per cluster, a store-conditional may only consume the reservation if it owns it.
4) **The non-coherent baseline never withdrawing a granted request.** A request must be dropped on grant, not on response. The cached path had always done so; the baseline had not, and surplus reads returned data into whatever transaction was outstanding by then.
5) **Combinational blocks re-triggering on their own writes.** A bit-write through a variable index is a read-modify-write of the whole vector, which places the vector in its own block's sensitivity list. The block reschedules forever and simulation time stops, with no error and nothing visible in a waveform, because the same values are reassigned.
6) **A helper function reading a port rather than taking it as an argument**, so the port never entered the sensitivity list of a continuous assignment and every invalidation carried a stale address.

VII. Threats to Validity

Our claim in Section V-C is that a published result does not follow from its stated configuration, and it rests on our reading of that configuration. A different line size, a shared rather than private cache organisation, or a different definition of the FFT working set would each explain the discrepancy without any error in the original. We state our assumptions explicitly for that reason: 2 KiB per PE, private, two-way, 16-byte lines, an $8N$-byte per-PE working set for a complex single-precision FFT of size N.

The original targets a Virtex UltraScale+ XCVU9P at 60 MHz and this work targets an XCZU7EV at 75 MHz, so absolute resource and frequency figures are directional rather than a like-for-like comparison. Our processing element presents three AXI masters where the original's figure implies two, which inflates our crossbar's resource line relative to theirs.

Power is not reported. Vivado's vectorless estimate models this design as idle, because it cannot evaluate the clock-gate enables on the cores, and a switching-activity-based estimate has not yet been completed.

VIII. Conclusion

We have reproduced a seamless snoop-coherent data-cache extension for multi-core RISC-V in open-source RTL and implemented it on an FPGA, and in doing so found that the original's flat FFT hit-rate curve does not follow from its stated 2 KiB cache. The effect is one of capacity, not conflict: capacity outweighs associativity by $6.6{:}1$ and then $44{:}1$ as the working set grows, and additional capacity stops helping precisely when the working set fits. We further show that the extension's benefit is contingent on reuse and memory latency: it converts a latency-bound workload into a compute-bound one, and where there is neither reuse nor latency to hide it is slower than no cache at all. The design, data and regeneration scripts are available at https://github.com/Krishna-Gorai/riscv-cache-extension.

References

[1] A. Kamaleldin, M. Nickel, S. Wu and D. Göhringer, "Seamless Cache Extension for FPGA-based Multi-Core RISC-V SoC," in *Proc. IEEE 37th Int. System-on-Chip Conf. (SOCC)*, 2024, doi:10.1109/SOCC62300.2024.10737850.